

Power Query M

Practice Challenges Workbook

From Zero to PL-300 Certified — 30 Hands-On Challenges with Real Public Datasets

Level	Who It's For	PL-300 Domain	# Challenges
■ Beginner	No prior M experience	Data connectivity & transformation basics	6
■ Easy	Familiar with Power Query UI	Transform & clean data	7
■ Intermediate	Comfortable with basic M	Model data, relationships	7
■ Advanced	Writing M queries regularly	Optimize queries, parameters	6
■ Expert	PL-300 exam ready	Complex real-world scenarios	4

All exercises use **free, publicly available datasets** from sources including Kaggle, the World Bank, Our World in Data, and Microsoft's own sample data. Each challenge specifies the exact download URL.

How to Use This Workbook

- | | |
|---|--|
| 1. Download the dataset | Each challenge links to a free public dataset. Download it and load it into Power BI Desktop via Get Data → CSV/Excel/Web. |
| 2. Open Advanced Editor | In Power Query Editor, go to Home → Advanced Editor. This is where you'll write and test your M code. |
| 3. Read the Scenario & Tasks | Understand what business problem you're solving before writing any code. |
| 4. Use the Starter Code | Copy the starter code into the Advanced Editor as your scaffold. Fill in the blanks or modify as instructed. |
| 5. Compare your output | The Expected Output section describes what the result should look like. Validate your work. |
| 6. Check the Hints | Only read hints if you're stuck — try to solve it independently first! |
| 7. Read the Pro Tip | Every challenge has a PL-300 exam tip that connects the exercise to what you'll be tested on. |

Key Notation Used

// comment	Explanatory comment in M code (not executed)
each _	Shorthand for a function where _ is the current row/item
[]	Column reference in a row context, e.g., [Sales]
{ }	A list literal, e.g., {1, 2, 3}
#date(y,m,d)	Date literal, e.g., #date(2024, 1, 1)
? in type	Nullable type, e.g., nullable text means text or null

★ ■ LEVEL 1 — BEGINNER | Data Types, Loading & Basic Transformations ★

These six challenges teach you to load data, understand M query structure, change data types, rename columns, and apply simple filters. They map to the PL-300 domain: **Prepare Data (15–20%)**.

Challenge 1

Your First M Query — Load & Inspect World Population Data

BEGINNER

Dataset: World Population by Country (CSV)	PL-300 Skill: Connect to data sources / Identify data anomalies	M Functions: Table.TransformColumnTypes, Table.ColumnNames
---	--	---

■ **Data Source:** <https://raw.githubusercontent.com/datasets/population/master/data/population.csv>

■ OBJECTIVE

Understand the let...in structure, load a CSV from a URL, and inspect column types. This is the absolute starting point for every Power Query workflow.

■ SCENARIO

You work at a consultancy and your manager asks you to pull world population data into Power BI. Before any analysis, you need to ensure the data loads cleanly with correct types.

■ TASKS

1. Load the CSV from the URL above using Web connector or paste into Advanced Editor.
2. Use Table.ColumnNames() to list all columns — write the names in the box below.
3. Use Table.TransformColumnTypes to cast 'Year' to Int64.Type and 'Value' to type number.
4. Rename the 'Value' column to 'Population' using Table.RenameColumns.
5. Close and Apply. Verify the data loads in the data model.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = Csv.Document(
Web.Contents("https://raw.githubusercontent.com/datasets/population/master/data/population.csv"),
[Delimiter=",", Columns=4, Encoding=65001, QuoteStyle=QuoteStyle.None] ), // Step 1: Promote first
row as headers PromotedHeaders = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), // Step 2:
Change types – YOUR TURN: fill in the types ChangedTypes =
Table.TransformColumnTypes(PromotedHeaders, { {"Country Name", type text}, {"Country Code", type
text}, {"Year", Int64.Type}, {"Value", type number} // <-- rename this column next }), // Step 3:
Rename 'Value' to 'Population' Renamed = Table.RenameColumns(ChangedTypes, {"Value",
"Population"}) in Renamed
```

■ EXPECTED OUTPUT

Table with 4 columns: Country Name (text), Country Code (text), Year (integer), Population (number)
~15,000+ rows covering countries from 1960 to present. No null values in Year or Population
columns.

■ HINTS

→ Use Home → Advanced Editor to paste the starter code directly.

→ Table.PromoteHeaders treats row 1 as column names — always needed after Csv.Document.

→ Int64.Type and type number are both numeric but Int64 is whole numbers only.

■ **PRO TIP (PL-300):** PL-300 tests whether you know when to use `Int64.Type` vs type number vs type decimal. Rule: use `Int64.Type` for IDs and years, type number or `Currency.Type` for financial values.

Challenge 2

Filter Rows — Keep Only Countries with Population Over 50 Million

BEGINNER

Dataset: World Population by Country (from Challenge 1)

PL-300 Skill: Profile data / Filter rows

M Functions: `Table.SelectRows`, `Table.RowCount`

■ **Data Source:** <https://raw.githubusercontent.com/datasets/population/master/data/population.csv>

■ OBJECTIVE

Practice `Table.SelectRows` and conditional logic. Row filtering is one of the most frequent Power Query operations and a guaranteed PL-300 topic.

■ SCENARIO

Your manager wants to focus the dashboard only on major countries. Filter the dataset to keep only rows where Population exceeds 50 million AND the Year is 2020.

■ TASKS

1. Continue from Challenge 1's output or reload the data.
2. Add a step to filter for Year = 2020 only.
3. Then filter for Population > 50,000,000.
4. Add a `Table.AddIndexColumn` step to number the remaining rows from 1.
5. Use `Table.RowCount` to count how many countries qualify — the answer should be around 30.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let // Paste your Challenge 1 query here, then add steps below PreviousStep = /* your Challenge 1 output */
// Filter to year 2020 Year2020 = Table.SelectRows(PreviousStep, each [Year] = 2020), //
// Filter to population over 50 million – YOUR TURN: fill in the condition MajorCountries =
Table.SelectRows(Year2020, each [Population] > 50000000), // Add index starting at 1 WithIndex =
Table.AddIndexColumn(MajorCountries, "Rank", 1, 1, Int64.Type), // Count rows (this will show as a
number in the formula bar) CountRows = Table.RowCount(WithIndex) in WithIndex // Return the table,
not the count
```

■ EXPECTED OUTPUT

Filtered table with ~30 rows (countries with population > 50M in 2020). Columns: Country Name, Country Code, Year, Population, Rank. Rank column counts from 1 to ~30.

■ HINTS

→ `each [Year] = 2020` is shorthand for `(_) => _[Year] = 2020`.

→ You can chain multiple `SelectRows` steps or combine with 'and': `each [Year]=2020 and [Population]>50000000`.

→ `Table.RowCount` returns a number, not a table — don't return it in the 'in' clause.

■ **PRO TIP (PL-300):** PL-300 Question Type: "Which function returns only rows matching a condition?" — Answer: `Table.SelectRows`. Know it cold.

Challenge 3**Text Cleanup — Clean a Messy Product Names Dataset****BEGINNER****Dataset:** Superstore Sales Dataset (Kaggle)**PL-300 Skill:** Clean data / Transform column values**M Functions:**

Table.TransformColumns, Text.Trim, Text.Proper, Text.Replace

Data Source: <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final> (file: Sample - Superstore.csv)**OBJECTIVE**

Apply Text.Trim, Text.Proper, and Text.Replace to clean inconsistent text data — a core real-world skill tested both on PL-300 and in professional BI work.

SCENARIO

You've received Superstore sales data but the 'Product Name' column has extra spaces, inconsistent casing, and some product names contain dashes that should be spaces. Clean it up.

TASKS

1. Load the Superstore CSV and keep only: Order ID, Product Name, Category, Sales.
2. Apply Text.Trim to Product Name to remove leading/trailing spaces.
3. Apply Text.Proper to Product Name to fix casing.
4. Replace all '-' characters in Product Name with a space using Text.Replace.
5. Verify no product name starts or ends with a space.

STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = Csv.Document(File.Contents("C:\Downloads\Sample - Superstore.csv"), [Delimiter=","  
Encoding=1252, QuoteStyle=QuoteStyle.Csv]), Headers = Table.PromoteHeaders(Source,  
[PromoteAllScalars=true]), // Keep only needed columns Selected = Table.SelectColumns(Headers,  
{"Order ID","Product Name","Category","Sales"}), // Fix types Typed =  
Table.TransformColumnTypes(Selected, { {"Sales", type number}, {"Product Name", type text} }), //  
Clean Product Name – YOUR TURN: chain Trim, Proper, and Replace CleanedNames =  
Table.TransformColumns(Typed, { {"Product Name", each Text.Proper(Text.Trim(Text.Replace(_, "-", "  
"))), type text} }) in CleanedNames
```

EXPECTED OUTPUT

Product Name column is title-cased, no leading/trailing spaces, dashes replaced by spaces. Example:
" staples easy-grip pencils " → "Staples Easy Grip Pencils"

HINTS

→ Table.TransformColumns takes a list of {column, transformation function} pairs.

→ You can nest functions: Text.Proper(Text.Trim(Text.Replace(_, "-", " "))) works inside 'each'.

→ The _ symbol inside 'each' refers to the current cell value.

PRO TIP (PL-300): PL-300 expects you to know the difference between transforming a column in-place (Table.TransformColumns) vs. adding a new one (Table.AddColumn). Use TransformColumns when replacing, AddColumn when you want both versions.

Challenge 4

Date Intelligence — Extract Year, Month, and Quarter from Order Dates

BEGINNER

Dataset: Superstore Sales Dataset (from Challenge 3)	PL-300 Skill: Create date/time columns / Build a calendar table	M Functions: Date.Year, Date.Month, Date.Day, Date.QuarterOfYear, Date.MonthName, Date.DayOfWeekName
---	--	---

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Learn Date.Year, Date.Month, Date.QuarterOfYear, and how to build a calendar dimension — essential for every Power BI time-intelligence report.

■ SCENARIO

Your dashboard needs time intelligence. Add Year, MonthNumber, MonthName, Quarter, and DayOfWeek columns derived from the Order Date.

■ TASKS

1. Load Superstore, keep: Order ID, Order Date, Sales.
2. Convert Order Date to type date.
3. Add column 'Year' using Date.Year.
4. Add column 'MonthNum' using Date.Month.
5. Add column 'MonthName' using Date.MonthName.
6. Add column 'Quarter' using Date.QuarterOfYear — format as text like 'Q1', 'Q2', etc.
7. Add column 'DayOfWeek' using Date.DayOfWeekName.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load Superstore CSV */, Selected = Table.SelectColumns(Source, {"Order ID", "Order Date", "Sales"}), Typed = Table.TransformColumnTypes(Selected, {"Order Date", type date}, {"Sales", type number})), // Add Year AddYear = Table.AddColumn(Typed, "Year", each Date.Year([Order Date]), Int64.Type), // Add Month Number AddMonthNum = Table.AddColumn(AddYear, "MonthNum", each Date.Month([Order Date]), Int64.Type), // Add Month Name – YOUR TURN AddMonthName = Table.AddColumn(AddMonthNum, "MonthName", each Date.MonthName([Order Date]), type text), // Add Quarter – combine "Q" with the quarter number AddQuarter = Table.AddColumn(AddMonthName, "Quarter", each "Q" & Text.From(Date.QuarterOfYear([Order Date])), type text), // Add Day of Week – YOUR TURN: use Date.DayOfWeekName AddDOW = Table.AddColumn(AddQuarter, "DayOfWeek", each Date.DayOfWeekName([Order Date]), type text) in AddDOW
```

■ EXPECTED OUTPUT

Table with columns: Order ID, Order Date, Sales, Year (int), MonthNum (int), MonthName (text, e.g. "March"), Quarter (text, e.g. "Q1"), DayOfWeek (text, e.g. "Wednesday")

■ HINTS

→ Date.QuarterOfYear returns 1, 2, 3, or 4. Use Text.From() to convert to text before concatenating with "Q".

→ Date.MonthName uses the system locale by default. Add a culture param: Date.MonthName([Order Date], "en-US").

→ You can chain Table.AddColumn steps; each step builds on the previous.

■ **PRO TIP (PL-300):** On PL-300, a Date table must have a contiguous range of dates with no gaps. Always create a separate Date table (not just columns on a fact table) using List.Dates or a Calendar function.

Challenge 5**Conditional Column — Classify Sales as Low, Medium, or High****BEGINNER****Dataset:** Superstore Sales Dataset**PL-300 Skill:** Derive columns / Create conditional logic**M Functions:** Table.AddColumn, if-then-else (nested)**Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>**OBJECTIVE**

Use if-then-else in an AddColumn step to create a classification column — the M equivalent of Excel's nested IF. This tests logical reasoning within M expressions.

SCENARIO

The sales team wants to classify each order line as Low (< \$100), Medium (\$100–\$999), or High (≥ \$1,000) for quick dashboard filtering.

TASKS

1. Load Superstore, keep Order ID, Category, Sales.
2. Cast Sales to type number.
3. Add a 'SalesTier' column using nested if-then-else.
4. Verify the three tiers appear by using Table.Group to count rows per tier.

STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load Superstore */, Selected = Table.SelectColumns(Source, {"Order ID", "Category", "Sales"}), Typed = Table.TransformColumnTypes(Selected, {"Sales", type number}),  
// Add SalesTier column – YOUR TURN: complete the nested if  
AddTier = Table.AddColumn(Typed, "SalesTier", each if [Sales] >= 1000 then "High" else if [Sales] >= 100 then "Medium" else "Low", type text),  
// Verification: count rows per tier  
TierCount = Table.Group(AddTier, "SalesTier", {"Count", each Table.RowCount(_), Int64.Type}) in AddTier // Change to TierCount to verify
```

EXPECTED OUTPUT

AddTier table: SalesTier column contains only "Low", "Medium", or "High". TierCount (verification): 3 rows showing count per tier. Roughly Low: ~6000, Medium: ~3000, High: ~900.

HINTS

→ M evaluates if-then-else top to bottom. Put the MOST restrictive condition first (≥ 1000 before ≥ 100).

→ Return AddTier for the main output, or swap to TierCount to verify your buckets.

→ You can add a type annotation as the 4th parameter to Table.AddColumn: ..., type text

PRO TIP (PL-300): PL-300 distinguishes between creating conditional columns in M (query load time) vs DAX calculated columns (model time). Use M when the classification is based on raw source data; use DAX when it depends on model aggregations.

Challenge 6

Remove Duplicates & Nulls — Build a Clean Customer Table

BEGINNER

Dataset: Superstore Sales Dataset	PL-300 Skill: Identify and handle data anomalies / Profile data with column quality	M Functions: Table.Distinct, Table.SelectRows, Table.RowCount, Table.FillDown
--	--	--

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Use Table.Distinct and Table.SelectRows with null checks to produce a clean dimension table. Data quality is a major PL-300 topic.

■ SCENARIO

You need to build a Customers dimension table from the transactional data. The source has one row per order line, so Customer IDs repeat. You also notice some Customer Name values are null. Clean this up.

■ TASKS

1. Load Superstore, keep: Customer ID, Customer Name, Segment, Region.
2. Use Table.SelectRows to remove rows where Customer ID is null.
3. Use Table.FillDown on Customer Name to fill any nulls downward.
4. Use Table.Distinct to deduplicate by Customer ID.
5. Verify the final row count is ~793 unique customers.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load Superstore */, Selected = Table.SelectColumns(Source, {"Customer ID", "Customer Name", "Segment", "Region"}), Typed = Table.TransformColumnTypes(Selected, { {"Customer ID", type text}, {"Customer Name", type text}, {"Segment", type text}, {"Region", type text} }), // Remove rows with null Customer ID NoNullID = Table.SelectRows(Typed, each [Customer ID] <> null), // Fill down Customer Name nulls FilledName = Table.FillDown(NoNullID, {"Customer Name"}), // Deduplicate — keep only first occurrence of each Customer ID Deduped = Table.Distinct(FilledName, {"Customer ID"}), // Count check FinalCount = Table.RowCount(Deduped) in Deduped
```

■ EXPECTED OUTPUT

Clean Customer dimension table with ~793 unique rows. No null values in Customer ID or Customer Name. 4 columns: Customer ID, Customer Name, Segment, Region.

■ HINTS

→ each [Column] <> null — use <> null, not != null. M uses <> for 'not equal'.

→ Table.Distinct takes an optional second parameter — a list of key columns. If omitted, ALL columns must match.

→ Table.FillDown only fills DOWN (forward). Use Table.FillUp if you want the opposite.

■ **PRO TIP (PL-300):** PL-300 tests data profiling: Column Quality shows % valid/error/empty. Enable it under View → Column Quality in Power Query. Know how to use it to find the right rows to fix or remove.

★ ■ LEVEL 2 — EASY | Lists, Records, Merging & Appending ★

These seven challenges introduce List and Record functions, table merging (joins), appending queries, and basic parameters. They map to PL-300 domain: **Model Data (30–35%)** and the Transform portion of Prepare Data.

Challenge 7

List Magic — Generate a Complete Date Table

EASY

Dataset: No external dataset — generated entirely in M	PL-300 Skill: Create a Date table / Enable time intelligence	M Functions: List.Dates, List.Numbers, #date, Duration.From, Table.FromList, Table.AddColumn, Date.*
---	---	---

■ **Data Source:** No download needed — pure M code

■ OBJECTIVE

Use List.Dates and List.Numbers to generate data programmatically. A proper Date table is mandatory for PL-300 and every professional Power BI model.

■ SCENARIO

Every Power BI model needs a contiguous Date table for time intelligence (YoY, MTD, etc.). Generate one from scratch in M covering 2015–2025.

■ TASKS

1. Use List.Dates to generate all dates from Jan 1, 2015 to Dec 31, 2025.
2. Convert the list to a single-column table.
3. Add columns: Year, MonthNum, MonthName, Quarter, WeekNum, DayOfWeek, IsWeekend.
4. Add an 'IsWeekend' logical column (true if Saturday or Sunday).
5. Mark this table as the Date table in Power BI (Modeling → Mark as date table).

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let // Generate list of all dates from 2015-01-01 to 2025-12-31
StartDate = #date(2015, 1, 1),
EndDate = #date(2025, 12, 31),
DayCount = Duration.Days(EndDate - StartDate) + 1,
DateList = List.Dates(StartDate, DayCount, #duration(1, 0, 0, 0)), // Convert list to table
DateTable = Table.FromList(DateList, Splitter.SplitByNothing(), {"Date"}),
Typed = Table.TransformColumnTypes(DateTable, {"Date", type date}), // Add calendar columns
AddYear = Table.AddColumn(Typed, "Year", each Date.Year([Date]), Int64.Type),
AddMonth = Table.AddColumn(AddYear, "MonthNum", each Date.Month([Date]), Int64.Type),
AddMName = Table.AddColumn(AddMonth, "MonthName", each Date.MonthName([Date], "en-US"), type text),
AddQtr = Table.AddColumn(AddMName, "Quarter", each "Q" & Text.From(Date.QuarterOfYear([Date])), type text),
AddWeek = Table.AddColumn(AddQtr, "WeekNum", each Date.WeekOfYear([Date]), Int64.Type),
AddDOW = Table.AddColumn(AddWeek, "DayOfWeek", each Date.DayOfWeekName([Date], "en-US"), type text),
AddDOWNum = Table.AddColumn(AddDOW, "DayNum", each Date.DayOfWeek([Date]), Int64.Type), // YOUR TURN: Add IsWeekend – true if DayNum = 5 (Sat) or 6 (Sun)
AddWeekend = Table.AddColumn(AddDOWNum, "IsWeekend", each [DayNum] >= 5, type logical) in AddWeekend
```

■ EXPECTED OUTPUT

Date table with 4,018 rows (2015-01-01 to 2025-12-31). Columns: Date, Year, MonthNum, MonthName, Quarter, WeekNum, DayOfWeek, DayNum, IsWeekend. IsWeekend = true for Saturdays (DayNum=5) and Sundays (DayNum=6).

■ HINTS

→ #duration(1, 0, 0, 0) = 1 day. Duration.Days(EndDate - StartDate) + 1 gives total count.

→ *Date.DayOfWeek* returns 0=Sunday, 1=Monday ... 6=Saturday by default. Adjust for *IsWeekend* logic.

→ To use as a Date table, the Date column must have no duplicates and no gaps.

■ **PRO TIP (PL-300):** PL-300 Exam: You **MUST** mark a table as the Date Table to use built-in DAX time intelligence (TOTALYTD, SAMEPERIODLASTYEAR, etc.). Do this in Modeling → Mark as Date Table. The key column must be of type Date with no nulls or duplicates.

Challenge 8

Merge Queries — Join Sales to Products

EASY

Dataset: Superstore Sales Dataset	PL-300 Skill: Combine queries / Design star schema	M Functions: Table.NestedJoin, Table.ExpandTableColumn, JoinKind.LeftOuter
--	---	---

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Practice `Table.NestedJoin` and `Table.ExpandTableColumn` to merge two related queries — equivalent to a SQL JOIN. This is fundamental to building star schema models.

■ SCENARIO

You have separate Sales and Products tables. Join them on Product ID to enrich the sales data with product category and sub-category information.

■ TASKS

1. Create a 'Sales' query: Order ID, Product ID, Sales, Quantity, Profit.
2. Create a 'Products' query: Product ID, Product Name, Category, Sub-Category. Use Table.Distinct to deduplicate.
3. Use Table.NestedJoin to join Sales to Products on Product ID (Left Outer join).
4. Expand the nested table to bring in Category and Sub-Category.
5. Verify no nulls in Category after the join.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// ■■■ QUERY 1: Sales ■■■ let Source = /* Load Superstore CSV */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Sales = Table.SelectColumns(Headers, {"Order ID","Product ID","Sales","Quantity","Profit"}), SalesTyped = Table.TransformColumnTypes(Sales, { {"Sales", type number}, {"Quantity", Int64.Type}, {"Profit", type number}}) in SalesTyped // ■■■ QUERY 2: Products (reference query from same source) ■■■ let Source = /* Load Superstore CSV */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Prods = Table.SelectColumns(Headers, {"Product ID","Product Name","Category","Sub-Category"}), Distinct = Table.Distinct(Prods, {"Product ID"}) in Distinct // ■■■ QUERY 3: Joined ■■■ let SalesQ = Sales, // Reference to Query 1 ProdsQ = Products, // Reference to Query 2 // Nested join: left outer Joined = Table.NestedJoin(SalesQ, "Product ID", ProdsQ, "Product ID", "ProductDetails", JoinKind.LeftOuter), // Expand to get Category and Sub-Category – YOUR TURN Expanded = Table.ExpandTableColumn(Joined, "ProductDetails", {"Category", "Sub-Category"}, {"Category", "Sub-Category"}) in Expanded
```

■ EXPECTED OUTPUT

```
Joined table with columns: Order ID, Product ID, Sales, Quantity, Profit, Category, Sub-Category.
~9,994 rows. No null values in Category if all Product IDs match.
```

HINTS

→ `Table.NestedJoin` creates a new column containing a nested table — then you must expand it.

→ The 3rd and 4th params to `ExpandTableColumn` are the source column names and the output names.

→ `JoinKind.LeftOuter` keeps all Sales rows even if no matching Product exists.

■ **PRO TIP (PL-300):** PL-300 tests join types: `LeftOuter` (all from left), `RightOuter`, `Inner` (only matches), `FullOuter` (all from both). For star schemas, always use `LeftOuter` from the fact table to the dimension — you want all facts even if a dimension lookup fails.

Challenge 9

Append Queries — Combine Sales from Three Regional Files

EASY

Dataset: Superstore (split by Region) — simulate by filtering

PL-300 Skill: Combine data from multiple sources / Append queries

M Functions: `Table.Combine`, `Table.AddColumn`, `Table.SelectRows`

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Use `Table.Combine` (the M equivalent of Power Query's Append Queries) to stack data from multiple sources into one table. Critical for combining data from multiple files.

■ SCENARIO

Your company has four regional sales teams, each sending a monthly CSV. You receive them as separate queries. Combine them into one unified Sales table and add a Region tag.

■ TASKS

1. Simulate four regional tables by filtering Superstore by Region (East, West, Central, South).
2. Add a 'SourceRegion' text column to each filtered table before combining.
3. Use `Table.Combine` with a list of all four tables to stack them vertically.
4. Verify the combined row count equals the original (all rows present).
5. Ensure all column names are consistent across the four tables before combining.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let // Load full dataset Source = /* Load Superstore */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Base = Table.SelectColumns(Headers, {"Order ID", "Region", "Category", "Sales", "Order Date"}), Typed = Table.TransformColumnTypes(Base, {{"Sales", type number}, {"Order Date", type date}}), // Simulate regional "files" with filters East = Table.AddColumn(Table.SelectRows(Typed, each [Region]="East"), "SourceRegion", each "East", type text), West = Table.AddColumn(Table.SelectRows(Typed, each [Region]="West"), "SourceRegion", each "West", type text), Central = Table.AddColumn(Table.SelectRows(Typed, each [Region]="Central"), "SourceRegion", each "Central", type text), South = Table.AddColumn(Table.SelectRows(Typed, each [Region]="South"), "SourceRegion", each "South", type text), // Combine all four – YOUR TURN: list all four tables Combined = Table.Combine({East, West, Central, South}), // Verification RowCount = Table.RowCount(Combined) in Combined
```

■ EXPECTED OUTPUT

Combined table with ~9,994 rows (matches original source count). Includes all original columns plus SourceRegion. All four regions present in SourceRegion column.

■ HINTS

→ `Table.Combine` takes a list {} of tables. All tables must have identical column names and order.

→ If column names differ between files, use `Table.RenameColumns` on each before combining.

→ This is exactly what the Append Queries UI button does — it generates a `Table.Combine` step.

■ **PRO TIP (PL-300):** PL-300 scenario: 'You receive monthly files in a folder. How do you combine them automatically?' — Answer: Use a *Folder connector* and *Table.Combine* or the 'Combine Files' button. M handles this with *Table.Combine* under the hood.

EASY

Dataset: No external dataset — M-generated	PL-300 Skill: Create query parameters / Use dynamic values	M Functions: Record.Field, Record.FieldNames, Record.ToTable, List.Select
---	---	--

■ **Data Source:** *Pure M — no download needed*

■ OBJECTIVE

Use Record and List functions to build a configuration record that drives query behaviour. This advanced pattern is tested in PL-300 scenarios involving parameterised queries.

■ SCENARIO

You want a single M record to store configuration values (start date, target region, sales threshold) that other queries can reference — like a settings table.

■ TASKS

1. Create a Config record with fields: StartDate, EndDate, Region, MinSales.
2. Use Record.Field to extract each value by name.
3. Convert the Config record to a table using Record.ToTable.
4. Use the MinSales value from Config to filter a sales query dynamically.
5. Use Record.FieldNames to list all config keys.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// ■■ Config Query ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ let Config = [ StartDate
= #date(2020, 1, 1), EndDate = #date(2023, 12, 31), Region = "West", MinSales = 500 ], // YOUR TURN:
convert Config to a two-column table (Name / Value) AsTable = Record.ToTable(Config) in Config //
Return the record itself (other queries will reference it) // ■■ Sales Query using Config
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ let SalesBase = /* Load Superstore */, // Reference config
values MinSalesVal = Record.Field(Config, "MinSales"), // = 500 RegionVal = Record.Field(Config,
"Region"), // = "West" // Apply filters using config values – YOUR TURN Filtered =
Table.SelectRows(SalesBase, each [Sales] >= MinSalesVal and [Region] = RegionVal) in Filtered
```

■ EXPECTED OUTPUT

```
Config query: returns the record [StartDate, EndDate, Region, MinSales]. AsTable: 4-row table with columns Name and Value. Filtered Sales: only West region rows with Sales >= 500.
```

■ HINTS

→ *Record.Field(record, "FieldName")* is equivalent to *record[FieldName]* — both work in M.

→ To make Config query reusable, name it 'Config' and reference it from other queries.

→ *Record.ToTable* produces a table with exactly two columns: 'Name' and 'Value'.

■ **PRO TIP (PL-300):** PL-300: Parameters in Power BI are stored as M queries. The Manage Parameters UI generates code like: `'MinSales = 500 meta [IsParameterQuery=true, ...]'`. You can hand-code parameters using the same pattern.

Challenge 11

Text Parsing — Extract Structured Data from Free-Text Fields

EASY

Dataset: NYC Open Data — 311 Service Requests	PL-300 Skill: Parse and transform text columns	M Functions: Text.Split, Text.Contains, Text.Upper, Text.Length, Text.Middle, Text.PositionOf
--	---	--

■ **Data Source:** <https://data.cityofnewyork.us/api/views/erm2-nwe9/rows.csv?accessType=DOWNLOAD> (filter to ~10,000 rows)

■ OBJECTIVE

Use Text.Split, Text.Contains, Text.Middle, and Text.PositionOf to extract structured information from unstructured text columns — a common real-world ETL challenge.

■ SCENARIO

NYC 311 requests contain a 'Descriptor' field with free text. You need to extract the complaint category from the first word, check if 'NOISE' appears anywhere, and standardize values.

■ TASKS

1. Load the NYC 311 CSV (first 10,000 rows is enough).
2. Keep: Unique Key, Created Date, Complaint Type, Descriptor, Borough.
3. Add 'FirstWord': extract the first word of Descriptor using Text.Split.
4. Add 'IsNoise': true if 'NOISE' is in the uppercased Descriptor.
5. Add 'BoroughClean': trim and proper-case Borough.
6. Add 'DescLength': character count of Descriptor.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = Csv.Document(
Web.Contents("https://data.cityofnewyork.us/api/views/erm2-nwe9/rows.csv?accessType=DOWNLOAD"),
[Delimiter=",", Encoding=65001, QuoteStyle=QuoteStyle.Csv]), Headers = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]), Top10k = Table.FirstN(Headers, 10000), Selected =
Table.SelectColumns(Top10k, {"Unique Key","Created Date","Complaint Type","Descriptor","Borough"}),
// Extract first word from Descriptor AddFirstWord = Table.AddColumn(Selected, "FirstWord", each
Text.Split(Text.Trim([Descriptor]), " "){0}, type text), // YOUR TURN: Add IsNoise – true if
"NOISE" is in Text.Upper([Descriptor]) AddIsNoise = Table.AddColumn(AddFirstWord, "IsNoise", each
Text.Contains(Text.Upper([Descriptor]), "NOISE"), type logical), // Clean Borough AddBoroughClean =
Table.AddColumn(AddIsNoise, "BoroughClean", each Text.Proper(Text.Trim([Borough])), type text), //
Add description length AddLength = Table.AddColumn(AddBoroughClean, "DescLength", each
Text.Length([Descriptor]), Int64.Type) in AddLength
```

■ EXPECTED OUTPUT

Table with ~10,000 rows. New columns: FirstWord (text), IsNoise (logical), BoroughClean (properly cased), DescLength (integer). IsNoise = true for ~15-20% of rows depending on dataset snapshot.

■ HINTS

→ Text.Split returns a list. Use {0} to get the first element (0-indexed).

→ Always wrap Text.Split results in try...otherwise to handle empty strings gracefully.

→ Text.Trim + Text.Proper on Borough fixes 'BRONX' → 'Bronx' and 'brooklyn ' → 'Brooklyn'.

■ **PRO TIP (PL-300):** PL-300 real scenario: 'A column contains values like Address: 123 Main St, City: Austin. Extract the city.' Use Text.Middle with Text.PositionOf to locate the delimiter and extract the substring.

Challenge 12

Number Crunching — Calculate Profit Margin by Category

EASY

Dataset: Superstore Sales Dataset	PL-300 Skill: Aggregate and summarize data / Create calculated metrics	M Functions: Table.Group, List.Sum, List.Average, Number.Round, Number.Abs
--	---	---

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Use Number.Round, Number.Abs, and Table.Group with custom aggregators to build a profitability summary — bridging M aggregation with Power BI reporting needs.

■ SCENARIO

Finance wants a category-level summary showing Total Sales, Total Profit, Profit Margin %, and whether each category is profitable.

■ TASKS

1. Load Superstore, keep: Category, Sub-Category, Sales, Profit.
2. Use Table.Group to aggregate by Category: sum Sales, sum Profit.
3. Add a Margin% column: (TotalProfit / TotalSales) * 100, rounded to 2 decimals.
4. Add a 'IsProfitable' column: true if Margin% > 0.
5. Sort by Margin% descending.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load Superstore */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Selected = Table.SelectColumns(Headers, {"Category", "Sub-Category", "Sales", "Profit"}), Typed = Table.TransformColumnTypes(Selected, {"Sales", type number}, {"Profit", type number}), // Group by Category, aggregate Sales and Profit Grouped = Table.Group(Typed, {"Category"}, { {"TotalSales", each List.Sum([Sales]), type number}, {"TotalProfit", each List.Sum([Profit]), type number} }), // Add Margin% – YOUR TURN: (TotalProfit/TotalSales)*100 rounded to 2dp AddMargin = Table.AddColumn(Grouped, "MarginPct", each Number.Round([TotalProfit] / [TotalSales]) * 100, 2), type number), // Add IsProfitable AddFlag = Table.AddColumn(AddMargin, "IsProfitable", each [MarginPct] > 0, type logical), // Sort by margin descending Sorted = Table.Sort(AddFlag, {"MarginPct", Order.Descending}) in Sorted
```

■ EXPECTED OUTPUT

```
3-row summary table: Technology | ~$836K sales | ~$145K profit | ~17.4% | true Office Supp | ~$719K sales | ~$122K profit | ~17.0% | true Furniture | ~$742K sales | ~$18K profit | ~2.5% | true
```

■ HINTS

→ List.Sum([Sales]) inside Table.Group refers to the [Sales] column of the current group's sub-table.

→ Division by zero: wrap in try...otherwise if TotalSales could be 0.

→ Number.Round(x, 2) rounds to 2 decimal places. Use Number.RoundUp or Number.RoundDown for ceiling/floor.

■ **PRO TIP (PL-300):** PL-300 tip: Prefer DAX measures over M aggregated tables when metrics need to respond to slicer selections. Use M grouping for static summary tables that don't need interactivity.

Challenge 13

Append + Dedupe — Combine Multiple Year Files Into One Fact Table

EASY

Dataset: World Bank GDP Data (CSV per country group)	PL-300 Skill: Combine queries / Remove duplicates / Unpivot columns	M Functions: Table.Combine, Table.Distinct, Table.UnpivotOtherColumns, Table.SelectRows
---	--	--

■ **Data Source:** <https://data.worldbank.org/indicator/NY.GDP.MKTP.CD> → Download as CSV

■ OBJECTIVE

Combine, clean, and deduplicate a multi-sheet/multi-file dataset. Practicing with World Bank data builds real analyst skills used in economics, finance, and government BI work.

■ SCENARIO

The World Bank GDP CSV has one column per year (wide format). You need to unpivot it into a tall format (Country, Year, GDP) suitable for Power BI time-series charts.

■ TASKS

1. Load the World Bank GDP CSV (it has metadata rows at the top — skip them).
2. Remove non-data rows (header junk, empty rows).
3. Keep: Country Name, Country Code, and all year columns (1960 to 2022).
4. Unpivot all year columns into rows using Table.UnpivotOtherColumns.
5. Rename 'Attribute' to 'Year' and 'Value' to 'GDP_USD'.
6. Cast Year to Int64 and GDP_USD to number. Remove null GDP rows.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let // World Bank CSV has 4 junk lines at top – skip them Source =
Csv.Document(File.Contents("C:\Downloads\API_NY.GDP.MKTP.CD_DS2.csv"), [Delimiter=",",
Encoding=65001, QuoteStyle=QuoteStyle.Csv]), // Rows 1-4 are metadata – skip; row 5 is the real
header SkipRows = Table.Skip(Source, 4), Headers = Table.PromoteHeaders(SkipRows,
[PromoteAllScalars=true]), // Remove extra columns (Indicator Name etc.) – keep Country Name, Code
+ year cols Cleaned = Table.SelectColumns(Headers, List.Select(Table.ColumnNames(Headers), each _ =
"Country Name" or _ = "Country Code" or (try Number.From(_) >= 1960 otherwise false))), // Unpivot
year columns to rows – YOUR TURN Unpivoted = Table.UnpivotOtherColumns(Cleaned, {"Country Name",
"Country Code"}, "Year", "GDP_USD"), // Cast and clean Typed =
Table.TransformColumnTypes(Unpivoted, { {"Year", Int64.Type}, {"GDP_USD", type number}}), NoNulls =
Table.SelectRows(Typed, each [GDP_USD] <> null) in NoNulls
```

■ EXPECTED OUTPUT

Tall table with ~13,000+ rows. Columns: Country Name (text), Country Code (text), Year (int), GDP_USD (number). Each country-year combination is one row. No null GDP values.

■ HINTS

→ Table.Skip(Source, 4) skips the first 4 rows. World Bank CSVs always have metadata rows.

→ List.Select on ColumnNames lets you keep columns whose names pass a condition — here, numeric names >= 1960.

→ try Number.From(_) >= 1960 otherwise false — the 'try' prevents an error if the column name isn't numeric.

■ **PRO TIP (PL-300):** PL-300: Know when to pivot vs. unpivot. Wide format (one column per year) is human-readable but bad for Power BI. Tall format (one row per year) is what the data model needs for slicers and time intelligence.

★ ■ LEVEL 3 — INTERMEDIATE | Parameters, Functions & Advanced Transformations ★

These seven challenges introduce query parameters, custom M functions, List.Generate, Table.Buffer for performance, and multi-table star schema design. They map to PL-300 domains: **Model Data (30–35%)** and **Optimize Query Performance**.

Challenge 14	Custom Function — Reusable File Loader with Error Handling	INTERMEDIATE
Dataset: Any CSV (use Superstore or NYC 311)	PL-300 Skill: Create and use custom M functions / Handle errors	M Functions: (param) => expression, try...otherwise, Csv.Document, File.Contents

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Write a custom M function that wraps file loading with try...otherwise error handling. Custom functions are a key PL-300 topic and essential for professional M code.

■ SCENARIO

Your team loads many different CSV files. Instead of copying the same 5-step loading logic each time, you want one reusable function: `LoadCSV(filePath, delimiter)`.

■ TASKS

1. Create a new blank query named 'LoadCSV'.
2. Define it as a function taking: filePath (text) and delimiter (text).
3. Inside the function, load the CSV, promote headers, and return the table.
4. Wrap the whole thing in try...otherwise to return a friendly error message table on failure.
5. Call LoadCSV with the Superstore path and "," as arguments.

■ STARTER CODE (Paste into Power Query Advanced Editor)

[illegible]

■ EXPECTED OUTPUT

LoadCSV query returns a function (shows as 'Function' type in Power Query). Calling it with a valid path returns the Superstore table (~9994 rows). Calling it with a bad path returns a 1-row table: Error = "File not found or unreadable: ...".

■ HINTS

- | |
|---|
| → A function query returns a value of type function — it won't show row data until called. |
| → #table({"ColName"}, {"value"}) creates a quick inline table — useful for error fallback tables. |
| → try expression otherwise fallback — if expression errors, fallback is returned instead. |

■ **PRO TIP (PL-300):** PL-300: Custom functions shine in folder-based loading. When combining files from a folder, Power BI calls your function once per file. The function pattern (x) => let...in result is the core of all advanced M.

INTERMEDIA
TE

Dataset: No external dataset — M-generated	PL-300 Skill: Write advanced M expressions / Understand M evaluation model	M Functions: List.Generate, List.Accumulate, List.Transform, List.Select
---	---	---

■ **Data Source:** *Pure M*

■ OBJECTIVE

Master List.Generate for iterative computation — the M equivalent of a loop. This unlocks dynamic list creation, rolling calculations, and custom sequences.

■ SCENARIO

A financial model needs a sequence of growth projections based on a compounding rule. You'll model this with `List.Generate` — first on a simpler Fibonacci sequence, then on a compound growth series.

■ TASKS

1. Use List.Generate to produce the first 20 Fibonacci numbers.
2. Convert the list to a table with columns: N (index) and Fibonacci.
3. Use List.Generate to produce 10 years of compound growth: start \$100,000, grow 8% per year.
4. Add a 'Cumulative Growth %' column showing growth vs. the starting value.
5. Use List.Select to find which years the value exceeds \$200,000 (doubling).

■ STARTER CODE (Paste into Power Query Advanced Editor)

[illegible]

■ EXPECTED OUTPUT

GrowthTable: 10 rows. Year 1: ~\$108,000, Year 7: ~\$171,000, Year 9: ~\$199,900, Year 10: ~\$215,892.
GrowthPct at Year 10: ~115.9%. Doubling (> \$200,000) first occurs at Year 10 (8% rule: doubles ~every 9 years).

HINTS

→ *List.Generate* state with a record [Year=..., Value=...] is cleaner than using a list {prev, curr}.

→ The 4th parameter of `List.Generate` is the selector — it extracts what you want from the state.

→ `List.Select(GrowthList, each _ > 200000)` returns only values above the threshold.

■ **PRO TIP (PL-300):** PL-300 doesn't test `List.Generate` directly, but understanding iterative patterns is crucial for complex transformations. Examiners test whether you know M is a lazy functional language — expressions are only evaluated when needed.

Challenge 16		Parameters & Dynamic Queries — Filter by Date Range Parameter	INTERMEDIATE
Dataset: Superstore Sales Dataset	PL-300 Skill: Create and manage query parameters / Build dynamic filters	M Functions: Manage Parameters UI + M code, Date.From, Table.SelectRows	

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Create query parameters for StartDate and EndDate, then use them to dynamically filter a query. Parameters are a PL-300 must-know for building flexible, reusable reports.

■ SCENARIO

Your report needs to be refreshable with configurable date ranges. Users should be able to change the start and end date without editing any M code directly.

■ TASKS

1. Create two Date parameters: StartDate (default: 2020-01-01) and EndDate (default: 2023-12-31).
2. In the Sales query, filter Order Date between StartDate and EndDate.
3. Verify that changing the parameter values updates the filtered row count.
4. Add a 'DaysInRange' calculated step using `Duration.Days(EndDate - StartDate)`.
5. Bonus: Create a What-If style text parameter for Region and filter by it.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// ■■ Parameter Queries (create via Manage Parameters UI or manually) ■■ // StartDate parameter query: // #date(2020, 1, 1) meta [IsParameterQuery=true, Type="Date", IsParameterQueryRequired=true] // EndDate parameter query: // #date(2023, 12, 31) meta [IsParameterQuery=true, Type="Date", IsParameterQueryRequired=true] // ■■ Sales Query with Parameter Filter ██████████ let Source = /* Load Superstore */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Selected = Table.SelectColumns(Headers, {"Order ID","Order Date","Sales","Region"}), Typed = Table.TransformColumnTypes(Selected, [{"Order Date", type date}, {"Sales", type number}]), // Filter using parameters – reference by parameter query name Filtered = Table.SelectRows(Typed, each [Order Date] >= StartDate and [Order Date] <= EndDate), // Add informational step DaysInRange = Duration.Days(EndDate - StartDate), // Add a column showing which date range this data covers WithLabel = Table.AddColumn(Filterd, "DateRangeLabel", each Text.From(StartDate) & " to " & Text.From(EndDate), type text) in WithLabel
```

■ EXPECTED OUTPUT

```
Filtered table with rows only between StartDate and EndDate. With defaults (2020-2023):
~4,000-5,000 rows. DateRangeLabel column shows "2020-01-01 to 2023-12-31" on every row. Changing
StartDate to 2022-01-01 reduces rows significantly.
```

■ HINTS

→ Create parameters via Home → Manage Parameters → New Parameter in Power Query Editor.

→ Once created, parameters appear as queries in the Queries pane. Reference them by name.

→ Date parameters must match the column type. If Order Date is datetime, cast parameter to datetime.

■ **PRO TIP (PL-300):** PL-300: Parameters can be used for incremental refresh (RangeStart, RangeEnd) in Power BI Premium. The parameter names must be EXACTLY 'RangeStart' and 'RangeEnd' and typed as DateTime for incremental refresh to work.

Challenge 17

Table.Buffer — Optimize a Slow Cross-Join Query

INTERMEDIATE

Dataset: Superstore + generated lookup

PL-300 Skill: Optimize query performance / Use Table.Buffer

M Functions: Table.Buffer, Table.Join, Table.NestedJoin, Table.AddColumn

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Understand when and how to use Table.Buffer to prevent a table from being re-evaluated multiple times. Query optimization is a distinct PL-300 exam topic.

■ SCENARIO

You're joining a large fact table to a lookup table, but the lookup table itself is computed from a complex transformation. Without buffering, it re-evaluates on every row lookup. Buffer it for better performance.

■ TASKS

1. Create a 'ProductLookup' table (Product ID → Category mapping) from Superstore.
2. Wrap ProductLookup in Table.Buffer.
3. Join Sales to the buffered ProductLookup.
4. Add a step that intentionally references ProductLookup twice — note the performance difference.
5. Explain in comments why buffering helps in this scenario.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load Superstore */, Headers = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]), // ■■ Build the lookup table (simulates complex transformation) ■■
Products = Table.Distinct( Table.SelectColumns(Headers, {"Product ID","Category","Sub-Category"}),
{"Product ID"}), // Buffer the lookup – this forces evaluation ONCE and caches it // YOUR TURN: wrap
Products in Table.Buffer BufferedProducts = Table.Buffer(Products), // ■■ Main Sales table
Sales = Table.SelectColumns(Headers, {"Order ID","Product ID","Sales","Profit"}), // Join to the BUFFERED lookup (not Products directly) Joined
= Table.NestedJoin(Sales, "Product ID", BufferedProducts, "Product ID", "ProdInfo",
JoinKind.LeftOuter), Expanded = Table.ExpandTableColumn(Joined, "ProdInfo",
{"Category","Sub-Category"}) in Expanded // EXPLANATION (write your answer here): // Without
Table.Buffer: every row lookup re-runs the Products transformation. // With Table.Buffer: Products
is computed ONCE, held in memory, reused efficiently. // Use when: the same query is referenced in
multiple steps or joins.
```

■ EXPECTED OUTPUT

Joined table identical to Challenge 8 in structure. Performance: buffered version avoids re-evaluation of the Products transformation. In large models, this can reduce refresh time significantly.

■ HINTS

→ Table.Buffer(table) forces immediate evaluation and stores result in memory.

→ Only use Table.Buffer when the table is referenced multiple times. Unnecessary buffering wastes memory.

→ List.Buffer works the same way for lists. Both are part of Power Query's performance toolkit.

■ **PRO TIP (PL-300):** PL-300 Performance topic: If a query is slow, check whether: (1) filtering happens early (filter pushdown), (2) columns are removed early, (3) Table.Buffer is used appropriately. The exam tests all three concepts.

INTERMEDIA
TE

Dataset: Superstore Sales Dataset	PL-300 Skill: Design star schema / Create fact and dimension tables	M Functions: Table.SelectColumns, Table.Distinct, Table.AddIndexColumn, Table.RenameColumns, Table.Join
--	--	--

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Design and build a full star schema in Power Query: one Fact table and four Dimension tables. Star schema design is the single most important modeling concept on PL-300.

■ SCENARIO

Transform the flat Superstore CSV into a proper star schema with: FactSales, DimCustomer, DimProduct, DimGeography, and DimDate (from Challenge 7).

■ TASKS

1. Create DimCustomer: Customer ID (PK), Customer Name, Segment.
2. Create DimProduct: Product ID (PK), Product Name, Category, Sub-Category.
3. Create DimGeography: generate a surrogate key, City, State, Region, Postal Code.
4. Create FactSales: Order ID, Order Date, Customer ID (FK), Product ID (FK), Geography Key (FK), Sales, Quantity, Profit, Discount.
5. Verify all foreign keys in FactSales exist in their respective dimension tables.
6. Connect DimDate (from Challenge 7) via Order Date.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// DimCustomer let Source = /* Base table with all columns */, Dim = Table.Distinct( Table.SelectColumns(Source, {"Customer ID", "Customer Name", "Segment"}), {"Customer ID"}) in Dim // DimProduct let Source = /* Base table */, Dim = Table.Distinct( Table.SelectColumns(Source, {"Product ID", "Product Name", "Category", "Sub-Category"}), {"Product ID"}) in Dim // DimGeography (with surrogate key) let Source = /* Base table */, GeoBase = Table.Distinct( Table.SelectColumns(Source, {"City", "State", "Region", "Postal Code"})), // YOUR TURN: Add a surrogate key GeoKey starting at 1 WithKey = Table.AddIndexColumn(GeoBase, "GeoKey", 1, 1, Int64.Type) in WithKey // FactSales let Source = /* Base table */, Facts = Table.SelectColumns(Source, {"Order ID", "Order Date", "Customer ID", "Product ID", "City", "State", "Sales", "Quantity", "Profit", "Discount"}), // Join to DimGeography to get GeoKey WithGeo = Table.NestedJoin(Facts, {"City", "State"}, DimGeography, {"City", "State"}, "GeoRef", JoinKind.LeftOuter), Expanded = Table.ExpandTableColumn(WithGeo, "GeoRef", {"GeoKey"}), // Remove city/state (now represented by GeoKey) Final = Table.RemoveColumns(Expanded, {"City", "State"}) in Final
```

■ EXPECTED OUTPUT

4 clean dimension tables + 1 fact table. FactSales: ~9,994 rows, foreign keys: Customer ID, Product ID, GeoKey + Order Date. All FKs resolve to dimension rows (no orphaned keys). Connect in Model view: FactSales → DimCustomer (Customer ID), etc.

■ HINTS

→ A surrogate key is a generated integer ID — use `Table.AddIndexColumn` for this.

→ Join `FactSales` to `DimGeography` on `City+State` (composite key) to get the surrogate `GeoKey`.

→ After building the schema in Power Query, go to Model View to draw the relationships.

■ **PRO TIP (PL-300):** PL-300: Star schema vs. snowflake. PL-300 strongly prefers star schema (all dims directly related to fact). Snowflake (dims with sub-dims) causes longer DAX paths and slower performance. When in doubt — flatten your dimensions.

Challenge 19

Pivot & Unpivot — Build a Cross-Tab Sales Report

INTERMEDIATE

Dataset: Superstore Sales Dataset

PL-300 Skill: Reshape data / Pivot and unpivot columns

M Functions: `Table.Pivot`, `Table.Unpivot`, `Table.Group`, `List.Distinct`

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Use `Table.Pivot` and `Table.Unpivot` to reshape data for different report formats. Pivoting and unpivoting are frequently tested on PL-300 as data reshaping operations.

■ SCENARIO

Management wants a matrix showing total Sales by Region (rows) and Category (columns). Build this pivot table in M, then unpivot it back to verify you understand both directions.

■ TASKS

1. Group Superstore by Region + Category, sum Sales.
2. Use `Table.Pivot` to create a crosstab: Regions as rows, Categories as columns.
3. Use `List.Distinct` to dynamically get the list of unique categories (don't hard-code them).
4. Unpivot the pivoted table back to long format and confirm it matches the original aggregation.
5. Add a 'Total' column summing across all category columns.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load Superstore */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Selected = Table.SelectColumns(Headers, {"Region","Category","Sales"}), Typed = Table.TransformColumnTypes(Selected, {"Sales", type number}), // Step 1: Aggregate by Region + Category Grouped = Table.Group(Typed, {"Region","Category"}, { {"TotalSales", each List.Sum([Sales]), type number} }), // Step 2: Get unique categories dynamically (no hard-coding) Categories = List.Distinct(Grouped[Category]), // Step 3: Pivot Category column Pivoted = Table.Pivot(Grouped, Categories, "Category", "TotalSales", List.Sum), // Step 4: Add Total column - YOUR TURN: sum all category columns per row AddTotal = Table.AddColumn(Pivoted, "Total", each List.Sum(List.Transform(Categories, (cat) => Record.Field(_, cat))), type number) in AddTotal
```

■ EXPECTED OUTPUT

Pivot table: 4 rows (East/West/Central/South), 3 category columns + Total. Example: East | Furniture:~\$186K | Office Supplies:~\$176K | Technology:~\$211K | Total:~\$573K Column headers are dynamic — if a new category appears, it auto-adds.

■ HINTS

→ `Table.Pivot(table, values, attributeCol, valueCol, aggregator)` — aggregator `List.Sum` handles duplicates.

→ `List.Distinct(Grouped[Category])` extracts the `Category` column as a list then deduplicates.

→ `Record.Field(_, cat)` extracts the value of column 'cat' from the current row record (`_`).

■ **PRO TIP (PL-300):** PL-300: `Table.Pivot` in M is equivalent to using the `Pivot Column` button in the UI. Know both. Dynamic pivoting (using `List.Distinct` for column values) is advanced but tested in scenario questions.

Challenge 20

Error Handling at Scale — Clean a Dirty Financial Dataset

INTERMEDIATE

Dataset: FRED Economic Data — US GDP (CSV)

PL-300 Skill: Handle errors / Validate data quality

M Functions: `try...otherwise`, `Value.Is`, `Table.ReplaceErrorValues`, `Table.AddColumn`

■ **Data Source:** <https://fred.stlouisfed.org/graph/fredgraph.csv?id=GDP>

■ OBJECTIVE

Use `try...otherwise`, `Value.Is`, and `Table.AddColumn` with error catching to handle malformed values in a real economic dataset without the query failing.

■ SCENARIO

A financial dataset has some numeric columns with text entries ('N/A', '#REF!', empty strings). Casting to number will error on these. Handle them gracefully.

■ TASKS

1. Load the FRED GDP CSV.
2. Attempt to cast the `VALUE` column to type number — observe errors.
3. Use `Table.ReplaceErrorValues` to replace errors with null.
4. Add a 'IsValidNumber' column using `try Number.From([VALUE]) otherwise false`.
5. Add a 'CleanValue' column that returns the number if valid, otherwise 0.
6. Count rows with invalid values using `Table.RowCount + Table.SelectRows`.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = Csv.Document( Web.Contents("https://fred.stlouisfed.org/graph/fredgraph.csv?id=GDP"),
[Delimiter=",", Encoding=65001]), Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
// Try casting to number – this may produce errors on bad values AttemptCast =
Table.TransformColumnTypes(Headers, {{"VALUE", type number}}), // Replace any type errors with null
ReplaceErrors = Table.ReplaceErrorValues(AttemptCast, {{"VALUE", null}}), // YOUR TURN: Add
IsValidNumber – true if VALUE is not null AddIsValid = Table.AddColumn(ReplaceErrors,
"IsValidNumber", each [VALUE] <> null, type logical), // Add CleanValue – use number if valid, 0 if
null AddClean = Table.AddColumn(AddIsValid, "CleanValue", each if [VALUE] <> null then [VALUE] else
0, type number), // Count invalid rows InvalidCount = Table.RowCount(Table.SelectRows(AddIsValid,
each not [IsValidNumber])) in AddClean
```

■ EXPECTED OUTPUT

FRED GDP data is actually quite clean – most rows will be valid. `IsValidNumber` = true for all normal rows, false for any null/error rows. `CleanValue` = GDP value for valid rows, 0 for invalid. FRED data typically has one row per quarter from 1947 to present.

■ HINTS

→ `Table.ReplaceErrorValues` takes a list of {columnName, replacementValue} pairs.

→ `try Number.From([VALUE]) otherwise null` is safer than casting with `TransformColumnTypes` when data is dirty.

→ Combine with Value.Is: Value.Is([VALUE], type number) returns true if VALUE is already a number.

■ PRO TIP (PL-300): PL-300: Know the three ways to handle errors in Power Query: (1) Remove Error Rows, (2) Replace Errors (Table.ReplaceErrorValues), (3) Keep Error Rows for investigation. Each option is in the Home or Transform ribbon.

★ ■ LEVEL 4 — ADVANCED | Recursive Functions, Web APIs & Performance ★

Six advanced challenges covering recursive M functions, web API pagination, complex Table.Group aggregators, query folding, and building a fully dynamic report model. These map to PL-300: **Optimize Query Performance** and **Prepare Data — Advanced**.

Challenge 21	Web API — Load Paginated REST Data from the GitHub API	ADVANCED
Dataset: GitHub REST API — Public repository list	PL-300 Skill: Connect to web data sources / Handle pagination	M Functions: Web.Contents, Json.Document, List.Generate, Table.FromRecords, Record.Field

■ **Data Source:** https://api.github.com/users/microsoft/repos?per_page=100&page;=1 (public, no auth needed)

■ OBJECTIVE

Use Web.Contents with recursive List.Generate to handle paginated API responses — a critical real-world skill for any Power BI developer connecting to REST APIs.

■ SCENARIO

You need to pull all public Microsoft GitHub repositories. The API returns 100 results per page. You must loop through pages until no more results are returned.

■ TASKS

1. Use Web.Contents to fetch page 1 of the GitHub API response.
2. Parse the JSON response with Json.Document.
3. Use List.Generate to iterate through pages until an empty result is returned.
4. Combine all pages into one table.
5. Extract: repo name, description, language, star count, fork count, created date.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let BaseUrl = "https://api.github.com/users/microsoft/repos", // Function to fetch one page
FetchPage = (page as number) as list => let Url = BaseUrl & "?per_page=100&page;" &
Text.From(page), Response = Web.Contents(Url, [Headers=[ #"User-Agent"="PowerBI-M-Practice",
Accept="application/vnd.github.v3+json" ]]), Parsed = Json.Document(Response) in Parsed, // Returns
a list of records // YOUR TURN: Use List.Generate to collect all pages AllPages = List.Generate( ()
=> [Page=1, Data=FetchPage(1)], each List.Count([Data]) > 0, // continue while page has data each
[Page=[Page]+1, Data=FetchPage([Page]+1)], each [Data] // collect each page's data ), // Flatten
list of lists into one list AllRepos = List.Combine(AllPages), // Convert to table RepoTable =
Table.FromRecords(AllRepos), // Select and rename key columns Selected =
Table.SelectColumns(RepoTable, {"name","description","language","stargazers_count",
"forks_count","created_at"}), Renamed = Table.RenameColumns(Selected, { {"name","RepoName"},
{"stargazers_count","Stars"}, {"forks_count","Forks"}, {"created_at","Created"}}) in Renamed
```

■ EXPECTED OUTPUT

Table with 200-400+ rows (Microsoft has many public repos). Columns: RepoName, description, language, Stars, Forks, Created. Sorted by Stars descending, top repos include VS Code, TypeScript, etc.

■ HINTS

→ GitHub API requires a User-Agent header — without it you get a 403 error.

→ *Json.Document* returns a list of records for array responses. Each record is a *repo*.

→ *List.Generate* stops when *List.Count([Data]) = 0* (empty page). This is the pagination termination condition.

■ **PRO TIP (PL-300):** PL-300 Advanced: For APIs requiring OAuth, use Web.Contents with [Headers=[Authorization='Bearer ' & token]]. Never hard-code tokens — use Power BI parameters or key vault references. Query folding does NOT apply to Web.Contents sources.

Challenge 22Recursive Function — Flatten a Nested JSON Hierarchy			ADVANCED
Dataset: World Bank Country API (JSON)	PL-300 Skill: Handle complex JSON / Flatten nested structures	M Functions: @ (recursive), Record.FieldNames, Record.Field, List.Transform, Table.FromRecords	

■ **Data Source:** https://api.worldbank.org/v2/country?format=json&per_page=300 (JSON)

■ OBJECTIVE

Write a recursive M function to flatten nested records/lists of arbitrary depth. This is an expert-level M skill needed when working with complex JSON from APIs.

■ SCENARIO

The World Bank API returns deeply nested JSON with country region and income level nested inside each country record. Flatten this automatically using a recursive function.

■ TASKS

1. Fetch the World Bank country JSON.
2. Inspect the nested structure — region and incomeLevel are nested records.
3. Write a recursive FlattenRecord function that descends into nested records.
4. Apply it to produce a flat table of all countries with region and income fields.
5. Filter to only show countries (not aggregates) and sort by income level.

■ STARTER CODE (Paste into Power Query Advanced Editor)

[illegible]

■ EXPECTED OUTPUT

~200 country rows with flattened columns including: id, iso2Code, name, region_id, region_value, incomeLevel_id, incomeLevel_value, capitalCity, longitude, latitude.

■ HINTS

→ @FlattenRecord calls the function recursively. The @ symbol is required inside a let binding for self-reference.

→ Value.Is(val, type record) returns true when val is a nested record — this is the recursion trigger.

→ World Bank uses {0} for pagination metadata and {1} for the actual data array.

■ **PRO TIP (PL-300):** PL-300: JSON flattening is tested via the 'Expand' button in Power Query UI. Understanding the M behind it (Table.ExpandRecordColumn) makes you faster at diagnosing expand errors and gives you control over column naming.

Challenge 23

Query Folding — Maximize SQL Push-Down on a Database Source

ADVANCED

Dataset: AdventureWorks Sample Database (SQL Server / Azure SQL)

PL-300 Skill: Optimize query performance / Understand query folding

M Functions: Sql.Database, Table.SelectColumns, Table.SelectRows, Table.Sort, Table.Buffer (breaks folding)

■ **Data Source:** <https://learn.microsoft.com/sql/samples/adventureworks-install-configure> OR use Azure free tier

■ OBJECTIVE

Understand query folding — Power Query's ability to push transformations to the source as SQL. This is a top PL-300 exam topic for query performance optimization.

■ SCENARIO

You're connecting to AdventureWorks in SQL Server. Your goal is to write an M query that folds as many steps as possible to SQL, then deliberately break folding and observe the difference.

■ TASKS

1. Connect to AdventureWorks Sales.SalesOrderHeader table.
2. Filter rows where TotalDue > 1000 — verify this folds (right-click step → View Native Query).
3. Select only: SalesOrderID, OrderDate, CustomerID, TotalDue.
4. Sort by OrderDate descending — verify folding holds.
5. Add a Table.Buffer step after sort — verify this BREAKS folding for subsequent steps.
6. Add a custom column after Buffer — note it runs in M, not SQL.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let // Connect to SQL Server – FOLDS to SQL Source =  
Sql.Database("your-server.database.windows.net", "AdventureWorks"), Table =  
Source[[Schema="Sales",Item="SalesOrderHeader"]][Data], // Step 1: Filter – FOLDS (WHERE clause in  
SQL) Filtered = Table.SelectRows(Table, each [TotalDue] > 1000), // Step 2: Select columns – FOLDS  
(SELECT clause) Selected = Table.SelectColumns(Filterd,  
{"SalesOrderID","OrderDate","CustomerID","TotalDue"}), // Step 3: Sort – FOLDS (ORDER BY clause)  
Sorted = Table.Sort(Selected, {"OrderDate", Order.Descending}), // !! This BREAKS query folding –  
everything after this runs in M engine !! Buffered = Table.Buffer(Sorted), // Step 4: Custom column  
– runs in M (no SQL equivalent) // YOUR TURN: Add a Tier column using if-then-else AddTier =  
Table.AddColumn(Buffered, "Tier", each if [TotalDue] >= 10000 then "High" else if [TotalDue] >=  
5000 then "Medium" else "Low", type text) in AddTier
```

■ EXPECTED OUTPUT

Table loaded from SQL with WHERE, SELECT, ORDER BY pushed to the database. Steps 1-3 show 'View Native Query' option (= folding active). After Table.Buffer, 'View Native Query' is greyed out (= folding broken). Final table has SalesOrderID, OrderDate, CustomerID, TotalDue, Tier.

■ HINTS

→ Right-click any query step → View Native Query to see the SQL being generated. If greyed out, folding broke.

→ Steps that break folding: Table.Buffer, Table.AddIndexColumn, adding columns with complex M logic, using List functions.

→ Steps that usually fold: SelectRows, SelectColumns, Sort, RenameColumns, TransformColumnTypes.

■ **PRO TIP (PL-300):** PL-300 key rule: Filter and Select columns as **EARLY** as possible in your query. This maximizes folding and minimises data transferred from the source. A query that folds a filter from 10M rows to 100K rows is 100x faster than loading all 10M rows and filtering in M.

Challenge 24

Dynamic Column Selection — Build a Flexible Unpivot Query

ADVANCED

Dataset: Our World in Data — COVID-19 Dataset

PL-300 Skill: Build resilient, schema-flexible queries

M Functions: Table.ColumnNames, List.Select, Table.UnpivotOtherColumns, List.Contains

■ **Data Source:** <https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv>

■ OBJECTIVE

Use List.Select, Table.ColumnNames, and dynamic column lists to build unpivot queries that automatically adapt to schema changes — avoiding hard-coded column names.

■ SCENARIO

The OWID COVID dataset has 60+ columns. You need to unpivot only the metric columns (those containing specific keywords like 'total', 'new', 'per') while keeping identifier columns static.

■ TASKS

1. Load the OWID COVID CSV (use first 50,000 rows for performance).
2. Dynamically identify 'id columns': iso_code, continent, location, date.
3. Dynamically identify 'metric columns': any column NOT in the id column list.
4. Unpivot only the metric columns using Table.UnpivotOtherColumns.
5. Filter to remove rows where Value is null.
6. Add a column categorising each metric as 'Deaths', 'Cases', 'Vaccinations', or 'Other'.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = Csv.Document( Web.Contents("https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv"), [Delimiter=";", Encoding=65001, QuoteStyle=QuoteStyle.Csv]),
Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Top50k = Table.FirstN(Headers, 50000), // Define the 'anchor' columns that stay as columns (not unpivoted) IdCols = {"iso_code","continent","location","date"}, // Dynamically identify metric columns (all others) AllCols = Table.ColumnNames(Top50k), // YOUR TURN: filter AllCols to exclude IdCols MetricCols = List.Select(AllCols, each not List.Contains(IdCols, _)), // Cast date column Typed = Table.TransformColumnTypes(Top50k, {"date", type date}), // Unpivot metric columns Unpivoted = Table.UnpivotOtherColumns(Typed, IdCols, "Metric", "Value"), // Remove null values NoNulls = Table.SelectRows(Unpivoted, each [Value] <> null and [Value] <> ""), // Cast Value to number
```

```
NumericVal = Table.TransformColumnTypes(NoNulls, {"Value", type number}), // Categorise metric
WithCat = Table.AddColumn(NumericVal, "Category", each if Text.Contains([Metric], "death") then
"Deaths" else if Text.Contains([Metric], "case") or Text.Contains([Metric], "new_cases") then
"Cases" else if Text.Contains([Metric], "vacc") then "Vaccinations" else "Other", type text) in
WithCat
```

■ EXPECTED OUTPUT

Tall table with columns: iso_code, continent, location, date, Metric, Value, Category. Rows: millions (50k source rows × ~60 metrics each, minus nulls). Category column groups metrics into Deaths/Cases/Vaccinations/Other.

■ HINTS

→ *List.Contains(list, value) returns true if value exists in list — use with List.Select for exclusion.*

→ *Table.UnpivotOtherColumns(table, idCols, attrName, valueName) — the second parameter is what you KEEP.*

→ *Dynamic column selection (no hard-coding) makes your query resilient when new columns are added to the source.*

■ **PRO TIP (PL-300):** PL-300 expert scenario: 'Your CSV source sometimes has extra columns. Your query errors when the schema changes.' Solution: Use dynamic column detection with Table.ColumnNames + List.Select rather than hard-coding column names in SelectColumns.

Challenge 25

Table.Group with Custom Aggregators — Rolling 7-Day Average

ADVANCED

Dataset: Our World in Data — COVID-19 (from Challenge 24)

PL-300 Skill: Advanced aggregation / Rolling calculations

M Functions: Table.Group, Table.Sort, List.Average, List.LastN, Table.AddIndexColumn

■ **Data Source:** <https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv>

■ OBJECTIVE

Use Table.Group with a custom sub-table aggregator to compute a rolling 7-day average — a common time-series metric in business analytics.

■ SCENARIO

For COVID analysis, calculate a rolling 7-day average of new cases per country. This requires sorting within each group and computing a windowed average — only possible with a custom aggregator.

■ TASKS

1. Filter OWID data to one country (e.g., United States) and the new_cases column.
2. Sort by date ascending.
3. Use Table.Group by location, then within each group sort by date and compute a 7-day rolling average.
4. The rolling average at each row = average of last 7 values of new_cases.
5. Return the result as a flat table with: location, date, new_cases, RollingAvg7.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
let Source = /* Load OWID COVID data */, // Filter to USA, keep date and new_cases USA =
Table.SelectRows(Source, each [location] = "United States"), Selected = Table.SelectColumns(USA,
{"location","date","new_cases"}), Typed = Table.TransformColumnTypes(Selected, {"date", type
date}, {"new_cases", type number}), Sorted = Table.Sort(Typed, {"date", Order.Ascending}), //
Add row index for windowing Indexed = Table.AddIndexColumn(Sorted, "RowIndex", 0, 1, Int64.Type), //
Compute 7-day rolling average using Table.Group approach // YOUR TURN: Add a RollingAvg7 column //
```

```
Hint: at each row, take the previous 6 rows + current row, average new_cases WithRolling =
Table.AddColumn(Indexed, "RollingAvg7", (currentRow) => let idx = currentRow[RowIndex], window =
Table.SelectRows(Indexed, each [RowIndex] >= idx - 6 and [RowIndex] <= idx), avg =
List.Average(window[new_cases]) in Number.Round(avg, 1), type number) in
Table.RemoveColumns(WithRolling, "RowIndex")
```

■ EXPECTED OUTPUT

USA COVID table with columns: location, date, new_cases, RollingAvg7. RollingAvg7 smooths out daily spikes. First 6 rows have smaller windows (averaging 1-6 values instead of 7). Performance note: this is slow on large datasets – Table.Buffer the Indexed table first.

■ HINTS

→ The inner `Table.SelectRows on Indexed` is a self-join pattern — this is slow on large data.

→ For production, pre-calculate rolling averages in DAX (AVERAGEX with DATESINPERIOD) — it's faster.

→ *Number.Round(avg, 1)* rounds to 1 decimal — *RollingAvg7* values like 45,231.3 are expected.

■ **PRO TIP (PL-300):** PL-300: Rolling averages in M are possible but slow. The exam tests whether you know that DAX is better for time-series calculations. In M: transform and clean data. In DAX: aggregate, calculate, and time-intelligence.

Challenge 26Function Invocation — Load All Files From a Folder DynamicallyADVANCED		
Dataset: Create 3 CSV files from Superstore (one per year: 2020, 2021, 2022)	PL-300 Skill: Combine files from folder / Automate file ingestion	M Functions: Folder.Files, Table.AddColumn, Function invocation, Table.Combine, Binary.Buffer

■ **Data Source:** Split from: <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Use the Folder connector and a custom invocation function to dynamically load and combine all CSV files from a folder — the professional standard for monthly/yearly file ingestion.

■ SCENARIO

Your company saves monthly sales CSVs to a shared folder. New files are added each month. Build a query that automatically picks up all CSV files from the folder and combines them.

TASKS

1. Save 3 Superstore-filtered CSVs (2020, 2021, 2022 data) to a local folder.
2. Use the Folder connector to get the file list.
3. Create a custom function that loads one CSV and adds a FileName column.
4. Use Table.AddColumn to invoke the function on each file's Binary content.
5. Expand the results and combine into one table.
6. Verify that adding a new file to the folder automatically picks it up on refresh.

■ STARTER CODE (Paste into Power Query Advanced Editor)

[illegible]

```
CsvOnly = Table.SelectRows(Source, each Text.EndsWith([Name], ".csv")), // Add column invoking the
function on each file's binary content // YOUR TURN: invoke TransformFile on the [Content] column
WithTables = Table.AddColumn(CsvOnly, "Data", each TransformFile([Content])), // Keep only filename
and data columns Selected = Table.SelectColumns(WithTables, {"Name","Data"}), // Expand the nested
tables Expanded = Table.ExpandTableColumn(Selected, "Data", Table.ColumnNames(Selected[Data]{0})),
// Add source file name column WithSource = Table.AddColumn(Expanded, "SourceFile", each [Name],
type text), // Remove the Name column (duplicated by SourceFile) Final =
Table.RemoveColumns(WithSource, {"Name"}) in Final
```

■ EXPECTED OUTPUT

Combined table from all CSV files in folder. SourceFile column shows which file each row came from. Adding a 4th CSV to the folder and refreshing automatically includes it. Columns match the CSV structure + SourceFile column.

■ HINTS

→ *Folder.Files()* returns a table with columns: Content (binary), Name, Date accessed, etc.

→ *Table.ColumnNames(Selected[Data]{0})* dynamically gets column names from the first table in the list.

→ *If files have different schemas, Table.Combine will add null for missing columns.*

■ **PRO TIP (PL-300):** PL-300 Folder ingestion exam scenario: 'You receive a new Excel file every month in the same format. How do you build the report to automatically include new files?' Answer: Folder connector + Combine Files button (or custom function pattern above).

★ ■ LEVEL 5 — EXPERT | PL-300 Exam Simulation Challenges ★

Four expert-level challenges that simulate real PL-300 exam scenarios. Each challenge combines multiple skills from all previous levels. These are the hardest exercises — attempt them only after completing Levels 1–4.

Challenge 27	Full ETL Pipeline — World Bank Development Indicators Dashboard	EXPERT
Dataset: World Bank Open Data API	PL-300 Skill: Full pipeline: ingest → transform → model → optimise	M Functions: All previously learned functions

■ **Data Source:** https://api.worldbank.org/v2/country/all/indicator/NY.GDP.MKTP.CD?format=json&per_page=500

■ OBJECTIVE

Build a complete, production-quality ETL pipeline: paginated API → cleaned data → star schema → parameterised date range. This combines all skills from Levels 1–4.

■ SCENARIO

Your BI team needs a World Bank GDP dashboard. You must: (1) pull data from the paginated API, (2) flatten nested JSON, (3) build a star schema, (4) add date parameters, (5) optimize with buffering.

■ TASKS

1. Fetch all pages of World Bank GDP data using List.Generate pagination.
2. Flatten the nested country and indicator JSON fields.
3. Build DimCountry (from Challenge 22 data) and DimDate (from Challenge 7).
4. Build FactGDP with countrycode, year, gdp_value as the fact table.
5. Add StartYear and EndYear parameters and filter FactGDP accordingly.
6. Buffer DimCountry and DimDate before joining to FactGDP.
7. Verify all FKs resolve and model relationships are correct.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// This is a capstone challenge – no starter code provided. // Design your own solution using the
patterns from Challenges 7, 16, 17, 18, 21, 22. // Suggested query structure: // 1. RawGDP_API –
paginated API fetch (List.Generate, Challenge 21 pattern) // 2. GDPFlattened – flatten nested
fields (Challenge 22 pattern) // 3. DimCountry – distinct countries with region/income (Challenge
6/18 pattern) // 4. DimDate – generated date table (Challenge 7) filtered by parameters // 5.
FactGDP – year, country_code, gdp_value (Challenge 18 pattern) // 6. StartYear / EndYear – query
parameters (Challenge 16 pattern) // 7. Config – record with model settings (Challenge 10 pattern)
// Target schema: // FactGDP: country_code (FK), year (FK), gdp_usd // DimCountry: country_code
(PK), country_name, region, income_level // DimDate: year (PK), decade, is_crisis_year (2008-2009,
2020)
```

■ EXPECTED OUTPUT

Fully functional star schema with 3 tables. FactGDP: ~4,000 rows (all countries × years 1960-2023, minus nulls). DimCountry: ~200 rows (one per country). DimDate: ~64 rows (one per year 1960-2023). Parameters: changing StartYear/EndYear reduces FactGDP row count accordingly.

■ HINTS

→ Start with a working API call for page 1, then wrap in List.Generate for pagination.

→ World Bank returns {pagination_info, data_array} — use Source{1} to get the data.

→ Buffer DimCountry before the join to FactGDP — it will be referenced multiple times.

■ **PRO TIP (PL-300):** PL-300 capstone skill: After building the model, check: (1) All relationships are set in Model View, (2) Date table is marked as Date Table, (3) Column directions are correct (single direction for star schema), (4) Hide FK columns in fact table from report view.

Challenge 28	Incremental Refresh Setup — Configure M for Premium Incremental Load	EXPERT
Dataset: Superstore Sales (simulate large dataset by repeating rows)	PL-300 Skill: Configure incremental refresh / Optimize large dataset refresh	M Functions: DateTime parameters, Table.SelectRows, Date.From

■ **Data Source:** <https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>

■ OBJECTIVE

Configure the exact M query structure required for Power BI Premium incremental refresh — using mandatory RangeStart/RangeEnd parameters with DateTime type.

■ SCENARIO

Your sales table has 10M+ rows and full refresh takes 2 hours. Configure incremental refresh to only reload the last 3 months of data while keeping the historical data intact.

■ TASKS

1. Create parameters named exactly 'RangeStart' and 'RangeEnd' of type DateTime.
2. In the Sales query, filter Order Date between RangeStart and RangeEnd.
3. Verify the filter uses >= and < (not <=) on the end date to avoid overlap.
4. Set up incremental refresh policy: store 3 years, refresh last 3 months.
5. Add a 'RefreshKey' column showing the date the row was last refreshed.
6. Document which steps will fold to the source vs. run in M.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// ■■ RangeStart Parameter (MUST be named exactly this) ■■■■■■ // Type: Date/Time (not Date – must be DateTime for incremental refresh) // Default value: 1/1/2020 12:00:00 AM // RangeStart = #datetime(2020, 1, 1, 0, 0, 0) // meta [IsParameterQuery=true, Type="DateTime", IsParameterQueryRequired=true] // ■■ RangeEnd Parameter ■■■■■■ // RangeEnd = #datetime(2023, 12, 31, 23, 59, 59) // meta [IsParameterQuery=true, Type="DateTime", IsParameterQueryRequired=true] // ■■ Sales Query with Incremental Filter ■■■■■■ let Source = /* Load Superstore */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), // Cast Order Date to datetime (required for incremental refresh filter) Typed = Table.TransformColumnTypes(Headers, {"Order Date", type datetime}), // Filter using EXACTLY RangeStart and RangeEnd // YOUR TURN: add the filter – use >= RangeStart and < RangeEnd Filtered = Table.SelectRows(Typed, each [Order Date] >= RangeStart and [Order Date] < RangeEnd), // Cast back to date for the model AsDate = Table.TransformColumnTypes(Filtered, {"Order Date", type date}) in AsDate
```

■ EXPECTED OUTPUT

Query filters correctly using RangeStart and RangeEnd. When RangeStart=2023-01-01 and RangeEnd=2023-04-01: only Q1 2023 rows returned. Parameters use DateTime type (not Date) – this is mandatory for incremental refresh. Filter uses < RangeEnd (not <=) to prevent duplicate loading of boundary day.

■ HINTS

→ Parameter names **MUST** be RangeStart and RangeEnd (exact capitalisation) for incremental refresh.

→ Type **MUST** be `DateTime` (not `Date`) — even if your column is `Date`, cast to `datetime` in the filter step.

→ Use `< RangeEnd` (strictly less than) so the last second of the period isn't double-loaded.

■ **PRO TIP (PL-300):** PL-300 Premium feature: Incremental refresh requires Power BI Premium or Premium Per User. Partition parameters RangeStart/RangeEnd are injected by Power BI Service at refresh time. Never hard-code these values — leave them at sensible defaults for local testing only.

Challenge 29	Composite Model — Combine DirectQuery & Import in One Dataset	EXPERT
Dataset: AdventureWorks SQL + Superstore CSV	PL-300 Skill: Design composite models / Mixed storage modes	M Functions: Sql.Database (DirectQuery), Csv.Document (Import), Table.Buffer, relationship configuration

■ **Data Source:** SQL: <https://learn.microsoft.com/sql/samples/adventureworks-install-configure> | CSV: Kaggle

■ OBJECTIVE

Design and build a composite model combining DirectQuery (for a large SQL fact table) and Import mode (for smaller dimension tables and a flat-file source). Composite models are a PL-300 advanced topic.

■ SCENARIO

Your company has a 50M-row SQL fact table that must stay in DirectQuery mode for live data. You also have smaller dimension tables in CSVs that should be imported for performance. Combine them.

■ TASKS

1. Connect to AdventureWorks SalesOrderHeader in DirectQuery mode.
2. Import Superstore Customer and Product CSVs as Import mode tables.
3. Create a bridge relationship between the two sources via a common key (simulate with Customer ID format matching).
4. Verify that slicing the Import dimension filters the DirectQuery fact table.
5. Identify which M steps will fold in DirectQuery vs. which break folding.
6. Document the dual-storage mode strategy in query comments.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// ■■■ DirectQuery Fact Table (AdventureWorks) ■■■■ // Set connection to
DirectQuery in Power BI Desktop: // Get Data → SQL Server → [connection] → DirectQuery option let
// This query folds entirely to SQL in DirectQuery mode Source = Sql.Database("your-server",
"AdventureWorks", [CreateNavigationProperties=false]), Orders =
Source[Schema="Sales",Item="SalesOrderHeader"][Data], // These steps FOLD to SQL (remain in
DirectQuery): Filtered = Table.SelectRows(Orders, each [TotalDue] > 0), Selected =
Table.SelectColumns(Filterd, {"SalesOrderID","OrderDate","CustomerID","TotalDue"}) // NOTE: Do NOT
add Table.Buffer or complex M steps in DirectQuery mode // They break folding and cause all data to
be pulled to M engine in Selected // ■■■ Import Dimension (Superstore Customers CSV)
■■■■ // Set this query to Import mode in Power BI Desktop let Source = /* Load
Superstore CSV */, Headers = Table.PromoteHeaders(Source, [PromoteAllScalars=true]), Customers =
Table.Distinct( Table.SelectColumns(Headers, {"Customer ID","Customer Name","Segment","Region"}),
{"Customer ID"}), // Buffer is OK in Import mode Buffered = Table.Buffer(Customers) in Buffered //
YOUR TURN: Document in comments: // 1. Which queries are DirectQuery vs Import // 2. What the
```

relationship between them is // 3. Why you chose Import for dimensions

■ EXPECTED OUTPUT

Two queries with different storage modes visible in Model View. DirectQuery fact table: live SQL with query folding. Import dimension: cached in memory, fast slicer performance. Relationship arrows in Model View show cross-source join. Slicing by Segment in Import dim filters the DirectQuery fact in real time.

■ HINTS

→ Set storage mode per table in Model View → right-click table → Storage Mode.

→ Dual mode (Import + DirectQuery) tables act as bridge tables in composite models.

→ Avoid complex M in DirectQuery tables — each visual query generates a SQL statement.

■ **PRO TIP (PL-300):** PL-300: Composite models let you mix DirectQuery and Import in one dataset. Key rule: Import dimensions improve performance for slicers. DirectQuery is best for very large fact tables needing real-time data. The exam tests when to use each mode.

Challenge 30

FINAL BOSS — End-to-End PL-300 Simulation: Multi-Source Analytics Model

EXPERT

Dataset: Superstore (CSV) +
World Bank GDP API +
Generated Date Table

PL-300 Skill: ALL PL-300 domains —
this is the capstone challenge

M Functions: ALL functions from the
reference guide

■ **Data Source:** Kaggle Superstore + api.worldbank.org + Pure M

■ OBJECTIVE

Build a complete, production-grade Power BI model from scratch using everything you have learned. This is the final PL-300 simulation challenge.

■ SCENARIO

Your consulting firm needs a C-suite dashboard showing: 1. Company sales performance (Superstore) by region and category over time. 2. Country-level GDP context for each region's primary market. 3. Full time intelligence (YoY growth, QTD, MTD). 4. Clean star schema with no performance issues. 5. Dynamic date range via parameters. Build the complete data model in Power Query.

■ TASKS

1. Build FactSales from Superstore with all required FK columns (Challenge 18).
2. Build DimCustomer, DimProduct, DimGeography from Superstore (Challenge 18).
3. Build DimDate spanning 2015–2025 with IsWeekend and IsHoliday columns (Challenge 7).
4. Build DimCountry_GDP from World Bank API with region and income level (Challenges 21-22).
5. Build FactGDP from World Bank with year and GDP per country (Challenge 27).
6. Add StartDate/EndDate parameters and filter both fact tables (Challenge 16).
7. Buffer all dimension tables used in multiple joins (Challenge 17).
8. Add query folding comments to every step explaining SQL vs M execution.
9. Verify: all relationships set, Date table marked, no circular dependencies.
10. Bonus: add a What-If parameter for Sales Threshold and document its use in DAX.

■ STARTER CODE (Paste into Power Query Advanced Editor)

```
// ██████████ // FINAL BOSS:
MULTI-SOURCE ANALYTICS MODEL // No starter code – design is yours. Use all patterns learned. //
██████████ // RECOMMENDED QUERY
LIST: // Parameters: StartDate, EndDate, SalesThreshold // Dimensions: DimDate, DimCustomer,
DimProduct, DimGeography, DimCountry // Facts: FactSales, FactGDP // Support: Config (settings
record), LoadCSV (helper function) // GRADING CHECKLIST (check each when done): // [ ] FactSales
has correct FK columns for all dimensions // [ ] DimDate is marked as Date Table in Power BI // [ ]
All date columns joined to DimDate via Order Date / Year // [ ] DimCountry joined to DimGeography
via Region (approximate mapping) // [ ] FactGDP filtered by StartDate/EndDate year parameters // [ ]
[ ] All dimensions buffered before join // [ ] No circular relationships in model // [ ] Hiding FK
columns from report view (do in Model View) // [ ] Adding a Date hierarchy in DimDate (Year >
Quarter > Month > Date) // [ ] Verify time intelligence works: YOY Sales measure in DAX // START
HERE – build one query at a time, test, then move to the next.
```

■ EXPECTED OUTPUT

Full data model with 7 queries: - 3 Parameters (StartDate, EndDate, SalesThreshold) - 5 Dimension tables (DimDate, DimCustomer, DimProduct, DimGeography, DimCountry) - 2 Fact tables (FactSales, FactGDP) - 2 Helper queries (Config, LoadCSV function) In Power BI Model View: - Star schema visible with all relationships - DimDate marked as Date Table - All FK columns hidden - No ambiguous relationships

■ HINTS

→ Build bottom-up: start with the smallest, cleanest dimension (DimDate), then build up to facts.

→ Test each query in isolation before referencing it from another query.

→ For DimCountry ↔ DimGeography join: map Region (Superstore) to World Bank region name — this requires a manual mapping table.

■ **PRO TIP (PL-300):** You've completed the full Power Query M mastery path. For PL-300 exam day: (1) Know star schema cold, (2) Know when M runs vs. when SQL folds, (3) Know all join types, (4) Know parameter types for incremental refresh, (5) Know when to use DAX vs. M for calculations. Good luck!

Appendix — PL-300 Quick Reference Card

Topic	Key M Function / Concept	PL-300 Weight
Load CSV from URL	<code>Csv.Document(Web.Contents(url))</code>	Medium
Load from SQL Server	<code>Sql.Database(server, db)</code>	High
Filter rows	<code>Table.SelectRows(t, each condition)</code>	High
Select columns	<code>Table.SelectColumns(t, {cols})</code>	High
Add column	<code>Table.AddColumn(t, name, each expr)</code>	High
Change types	<code>Table.TransformColumnTypes(t, {pairs})</code>	High
Group & aggregate	<code>Table.Group(t, keys, {aggregators})</code>	High
Join tables	<code>Table.NestedJoin</code> / <code>Table.Join</code>	High
Append tables	<code>Table.Combine({t1, t2, ...})</code>	Medium
Pivot	<code>Table.Pivot(t, values, attrCol, valCol)</code>	Medium
Unpivot	<code>Table.UnpivotOtherColumns(t, keepCols, ...)</code>	High
Remove duplicates	<code>Table.Distinct(t, optional keys)</code>	Medium
Fill down nulls	<code>Table.FillDown(t, {cols})</code>	Medium
Handle errors	<code>Table.ReplaceErrorValues</code> / <code>try...otherwise</code>	High
Query parameters	Manage Parameters → Type + Default	High
Incremental refresh	<code>RangeStart</code> / <code>RangeEnd</code> as DateTime params	High
Buffer for perf.	<code>Table.Buffer(t)</code>	Medium
Query folding	Filter/Select early; avoid M-only steps	High
Custom function	<code>(param as type) => let...in result</code>	High
Date table	<code>List.Dates</code> + mark as Date Table	Critical
Star schema	1 fact + multiple dims, single-dir relations	Critical
Storage mode	<code>Import</code> / <code>DirectQuery</code> / <code>Dual</code> / <code>Composite</code>	High
Recursive function	<code>@FunctionName</code> inside let binding	Low-Medium
Paginated API	<code>List.Generate</code> + <code>Web.Contents</code> loop	Medium

Recommended Study Order for PL-300

- Week 1-2** Complete Beginner challenges (1-6). Focus on let...in structure, types, and basic filters.
- Week 3-4** Complete Easy challenges (7-13). Master Date table, joins, appends, and unpivot.
- Week 5-6** Complete Intermediate challenges (14-20). Build full star schemas and use parameters.

Week 7-8	Complete Advanced challenges (21-26). API connectivity, query folding, performance.
Week 9	Complete Expert challenges (27-30). Full pipeline simulation.
Week 10	Review PL-300 exam guide, practice in Microsoft Learn sandbox, take mock exam.

Official PL-300 Study Resources: learn.microsoft.com/certifications/power-bi-data-analyst | docs.microsoft.com/power-query | community.powerbi.com